

Glossaire — Concepts clés du cours

Comment lire ce glossaire

Ce glossaire regroupe tous les concepts clés du cours, classés par thème. Chaque définition est volontairement courte — pour le détail, référez-vous à la session indiquée.

Mathématiques & Géométrie (Session 1)

Vecteur

Grandeur ayant une direction et une magnitude. Représenté par une flèche, décomposé en composantes selon les axes du plan cartésien.

Magnitude / Norme

Longueur d'un vecteur, notée $\|\vec{v}\|$. En 2D : $\sqrt{v_x^2 + v_y^2}$ (Pythagore).

Vecteur unitaire

Vecteur de longueur 1, noté $\hat{u}$. Obtenu en divisant un vecteur par sa norme : $\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$. Sert à représenter une direction pure.

Produit scalaire

Opération qui retourne un **nombre** : $\vec{u} \cdot \vec{v} = \|\vec{u}\| \cdot \|\vec{v}\| \cdot \cos(\theta)$. Nul si les vecteurs sont perpendiculaires.

Produit vectoriel

Opération 3D qui retourne un **vecteur** perpendiculaire aux deux opérandes : $\vec{a} \times \vec{b}$. Non commutatif. Sa norme donne l'aire du parallélogramme formé.

Projection

Composante d'un vecteur dans la direction d'un autre : $\text{proj}_{\vec{v}}(\vec{u}) = \left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{v}\|^2} \right) \vec{v}$.

Déterminant

Outil algébrique qui, pour le produit vectoriel, donne les composantes du vecteur perpendiculaire. Sert aussi à résoudre des systèmes d'équations.

Règle de Cramer

Méthode de résolution d'un système de n équations à n inconnues via les déterminants : $x = \frac{\Delta_x}{\Delta}$, etc.

Cinématique (Session 2)

Cinématique

Branche de la mécanique qui **décrit** le mouvement (position, vitesse, accélération) sans s'occuper de ses causes (forces).

Vecteur position

Vecteur $\vec{r}$ pointant de l'origine vers la position d'un objet. En 2D : $\vec{r} = \begin{pmatrix} x \\ y \end{pmatrix}$.

Vitesse

Dérivée de la position par rapport au temps. Vecteur tangent à la trajectoire. On distingue vitesse **moyenne** ($\Delta \frac{\vec{r}}{\Delta t}$) et **instantanée** ($d\frac{\vec{r}}{dt}$).

Accélération

Dérivée de la vitesse par rapport au temps. Peut changer la magnitude ou la direction de la vitesse (ou les deux).

Trajectoire

Courbe formée par l'ensemble des positions successives d'un objet au cours du temps.

Mouvement rectiligne uniforme

Mouvement en ligne droite à vitesse constante. Accélération nulle.

Mouvement circulaire uniforme

Mouvement sur un cercle à vitesse scalaire constante. L'accélération est centripète (vers le centre), de magnitude $R\omega^2$.

Mouvement parabolique (projectile)

Mouvement d'un objet soumis à la gravité seule. Vitesse horizontale constante, vitesse verticale décroissante. Accélération $\vec{a} = \begin{pmatrix} 0 \\ -g \end{pmatrix}$.

Lois de Newton (Session 3)

Première loi — Inertie

Un objet au repos reste au repos, un objet en mouvement continue en ligne droite à vitesse constante, **sauf** si une force extérieure agit.

Deuxième loi — Dynamique

$\vec{F} = m \cdot \vec{a}$. L'accélération est proportionnelle à la force et inversement proportionnelle à la masse.

Troisième loi — Action-Réaction

Pour toute force exercée par A sur B, B exerce une force égale et opposée sur A : $\vec{F}_{A \rightarrow B} = -\vec{F}_{B \rightarrow A}$.

Dynamique

Branche de la mécanique qui relie le mouvement aux **forces** qui le causent (contrairement à la cinématique).

Forces de la nature (Session 4)

Gravité / Poids

Force d'attraction vers le centre de la Terre : $\vec{P} = m \cdot \vec{g}$, avec $g \approx 9.81 \text{ "m/s"}^2$.

Force normale (N)

Force avec laquelle une surface repousse un objet qui appuie contre elle. Perpendiculaire à la surface. Sur un sol plat : $N = mg$.

Frottement sec (Coulomb)

Frottement de contact entre deux solides : $\|\vec{f}\| = \mu \cdot \|\vec{N}\|$. Dépend du coefficient de friction μ et de la normale.

Frottement statique

Frottement sur un objet **immobile**. S'adapte jusqu'à un seuil $F_{\max} = \mu_s N$, avec $\mu_s > \mu_k$ en général.

Frottement cinétique

Frottement sur un objet qui **glisse**. Force constante $F_k = \mu_k N$.

Frottement visqueux (fluide)

Résistance de l'air ou de l'eau. Linéaire ($-k \cdot \vec{v}$) à faible vitesse, quadratique ($-C \cdot \|\vec{v}\|^2 \cdot \hat{v}$) à haute vitesse.

Vitesse terminale

Vitesse limite atteinte quand la gravité et le frottement fluide s'annulent. L'objet ne peut plus accélérer.

Intégration numérique (Sessions 5, 8, 12)

Intégration numérique

Discretisation du temps continu en pas Δt pour calculer position et vitesse image par image. Nécessaire car l'ordinateur calcule frame par frame.

Pas de temps (dt)

Intervalle de temps entre deux frames. À 60 FPS, $\Delta t \approx 0.016$ s. Rend la physique indépendante du framerate.

Méthode d'Euler

Intégration du premier ordre : $\vec{v}_{n+1} = \vec{v}_n + \vec{a} \cdot \Delta t$, puis $\vec{r}_{n+1} = \vec{r}_n + \vec{v} \cdot \Delta t$. Simple mais dérive en énergie.

Euler semi-implicite (symplectique)

Variante où on met à jour la vitesse **avant** la position, et utilise la **nouvelle** vitesse. Plus stable, choix par défaut des moteurs de jeu.

Série de Taylor

Décomposition d'une trajectoire en ses dérivées successives. Euler ne garde que le premier terme (ordre 1).

Intégration de Verlet

Méthode qui ne stocke pas la vitesse explicitement : $v \approx \frac{x_n - x_{n-1}}{\Delta t}$. Plus précise ($O(\Delta t^4)$), symplectique, tolère les déplacements manuels de positions.

Damping (amortissement)

Dissipation d'énergie. En Verlet, on multiplie la vitesse implicite par un coefficient $d \leq 1$ à chaque frame.

RK4 (Runge-Kutta 4)

Intégration qui évalue l'accélération à **4 endroits** du pas de temps et fait une moyenne pondérée. Précision $O(\Delta t^4)$. Indispensable pour les orbites (systèmes conservatifs).

Sub-stepping

Diviser le Δt d'une frame en N sous-pas plus petits. Améliore la précision même avec un intégrateur simple. Complémentaire de RK4.

Système conservatif

Système sans dissipation (orbites, pendules). L'énergie se conserve — Euler y accumule de l'erreur, d'où RK4.

Impulsion & Collisions (Session 6)

Quantité de mouvement (p)

$\vec{p} = m \cdot \vec{v}$. Grandeur vectorielle qui mesure “combien de mouvement” porte un objet. Se conserve dans un système isolé.

Impulsion (J)

Produit $\vec{J} = \vec{F} \cdot \Delta t$ qui change la vitesse instantanément lors d'un choc : $\vec{J} = \Delta \vec{p} = m \cdot \Delta \vec{v}$. Évite de simuler une force énorme frame par frame.

Coefficient de restitution (e)

Rapport entre la vitesse de séparation et la vitesse d'approche : $v_{\text{rel}(\text{après})} = -e \cdot v_{\text{rel}(\text{avant})}$. $e = 1$: élastique ; $e = 0$: mou.

Vitesse relative

Vitesse de rapprochement projetée sur la normale de collision : $v_{\text{rel}} = (\vec{v}_A - \vec{v}_B) \cdot \vec{n}$. Négative = rapprochement.

Normale de collision

Direction perpendiculaire au contact, de A vers B. Seule la composante **normale** de la vitesse est modifiée par l'impulsion.

Masse réduite

$m_{\text{red}} = \frac{m_A m_B}{m_A + m_B}$. Combine les deux masses en une seule pour le calcul d'impulsion.

Vitesse tangentielle

Composante de vitesse perpendiculaire à la normale. Inchangée par l'impulsion (glissement).

Détection de collision — Broad Phase (Session 7)

Pipeline de collision

Entonnoir en 3 étapes : **Broad Phase** (candidats) → **Narrow Phase** (confirmation) → **Response** (impulsion). Chaque étage divise le travail par un ordre de grandeur.

Broad Phase

Première étape : trouver rapidement quelles paires d'objets **pourraient** entrer en collision, via des boîtes simplifiées (AABB). Élimine 95–99 % des paires.

Narrow Phase

Deuxième étape : tester géométriquement chaque paire candidate pour **confirmer** la collision et calculer le contact (point, normale, profondeur).

Collision Response

Troisième étape : appliquer l'impulsion et corriger la position suite à une collision confirmée.

AABB (Axis Aligned Bounding Box)

Plus petit rectangle/pavé **aligné sur les axes** contenant un objet. Définie par deux points min et max. Test de chevauchement trivial (6 comparaisons en 3D).

OBB (Oriented Bounding Box)

Comme une AABB mais **orientée** selon les axes locaux de l'objet. Plus précise pour les objets allongés et tournés. Testée via SAT (15 axes en 3D).

Grille uniforme (Spatial Hashing)

Découpe le monde en cases de taille fixe. Chaque objet va dans sa case. On ne teste que les 27 cases voisines (3D). Complexité $O(N)$ si distribution uniforme.

Quadtree / Octree

Arbre spatial : on découpe récursivement une boîte en 4 (2D) ou 8 (3D) sous-boîtes quand elle dépasse un seuil d'objets. S'adapte à la densité.

Sweep and Prune (SAP)

Projette les AABB sur un axe, trie les intervalles, maintient un ensemble "actif". Exploite la **cohérence temporelle** : quasi $O(N)$ amorti.

Cohérence temporelle

D'une frame à l'autre, les objets bougent peu. La liste triée de SAP est presque la même → tri par insertion $O(N)$ au lieu de $O(N \log N)$.

Narrow Phase avancée (Session 11)

SAT (Separating Axis Theorem)

Deux polygones convexes ne se touchent pas s'il existe un **axe séparateur** où leurs projections ne se chevauchent pas. On teste les normales des faces. L'axe au plus petit chevauchement donne le contact.

Axe séparateur

Axe sur lequel les projections de deux formes ont un **gap** : preuve qu'elles ne se touchent pas.

GJK (Gilbert-Johnson-Keerthi)

Algorithme itératif qui construit un **simplex** dans la **différence de Minkowski** $A - B$. Si l'origine est à l'intérieur $\rightarrow$ collision. Universel pour toute forme convexe.

Différence de Minkowski

Ensemble $A - B = \{a - b \mid a \in A, b \in B\}$. A et B se touchent ssi l'origine est à l'intérieur de $A - B$.

Fonction de support

$S_A(\vec{d})$ retourne le point de A le plus extrême dans la direction $\vec{d}$. Triviale pour les primitives (sphère, boîte).

Simplex

Généralisation du triangle : point (0D), segment (1D), triangle (2D), tétraèdre (3D). GJK le construit itérativement pour encercler l'origine.

EPA (Expanding Polytope Algorithm)

Prolonge GJK après collision : étend le simplex vers la vraie frontière de $A - B$ pour obtenir la **normale** et la **profondeur** de pénétration.

Contact

Triple (point P , normale $\hat{n}$, profondeur d) produit par la Narrow Phase, prêt pour la Response.

Position Based Dynamics (Session 8)

Position Based Dynamics (PBD)

Au lieu de calculer des forces (Hooke) puis d'intégrer, on **corrige directement les positions** pour satisfaire des contraintes géométriques. Inconditionnellement stable.

Contrainte de distance

Règle : “ p_1 et p_2 doivent être à distance L_0 ”. Si violée, on déplace chaque particule de la moitié de l’écart le long de la direction.

Itérations PBD

Nombre de passes de contraintes par frame. **La raideur émerge du nombre d’itérations** : $N = 1$ mou, $N = 20$ rigide. Pas de paramètre k .

Loi de Hooke

Force de rappel d’un ressort : $\vec{F} = -k \cdot (L - L_0) \cdot \hat{u}$. Instable avec Euler si k est grand — d’où PBD.

Shear constraint

Contrainte diagonale (en plus des horizontales/verticales) qui empêche un tissu de se déformer en losange.

Moteur de particules (Session 9)

Moteur de particules

Système qui simule un grand nombre de petits objets éphémères (pluie, feu, fumée) : émettre, mettre à jour, recycler.

Object Pool

Allouer une fois un tableau fixe de particules et **recycler** les mortes plutôt que d’en créer/détruire. Zéro allocation pendant le jeu — évite les saccades du garbage collector.

Emitter

Génère des particules à un taux donné, avec une forme (point, cône, sphère) qui contrôle position et direction initiales.

Drag linéaire (Stokes)

Force opposée à la vitesse, **proportionnelle** à v : $\vec{F} = -k \cdot \vec{v}$. Pour petits objets lents dans un fluide visqueux.

Drag quadratique

Force opposée à la vitesse, **proportionnelle** à v^2 : $\vec{F} = -k \cdot \|\vec{v}\| \cdot \vec{v}$. Pour gouttes de pluie, projectiles rapides.

Flottabilité (Archimède)

Force vers le haut égale au poids du fluide déplacé. Fusionnée en “gravité effective” $g_{\text{eff}} = g \left(1 - \frac{\rho_{\text{air}}}{\rho_{\text{particule}}} \right)$: négative = tombe, positive = monte.

Turbulence

Force pseudo-aléatoire (sinusoïdes déphasées) qui fait onduler la fumée et crépiter le feu. $O(1)$ par particule.

BufferGeometry

Conteneur de tableaux bruts (Float32Array) envoyés directement au GPU. Un attribut par propriété (position, color, size, alpha). Le seul format performant pour des milliers de particules.

THREE.Points

Objet Three.js qui dessine chaque sommet comme un **point** (carré de pixels) via `gl_PointSize`. Un seul draw call pour des milliers de particules.

Vertex shader

Programme GPU qui calcule la position projetée de chaque sommet (et, pour les points, la taille `gl_PointSize`).

Fragment shader

Programme GPU qui calcule la couleur finale de chaque pixel (ici : disque doux via `discard` + `smoothstep` sur `gl_PointCoord`).

Rotations & Corps rigides (Session 10)

Corps rigide

Objet avec étendue spatiale, centre de masse, orientation et répartition de masse. Peut **translater** et **pivoter** simultanément. Les points internes ne bougent pas les uns par rapport aux autres.

Particule vs corps rigide

Une particule est un point sans orientation. Un corps rigide a une étendue, une orientation et résiste à la rotation.

Couple (Torque)

Équivalent angulaire de la force : $\vec{\tau} = \vec{r} \times \vec{F}$. Dépend de la force **et** du bras de levier $\vec{r}$.

Bras de levier

Distance entre le centre de masse et le point d'application d'une force. Plus il est grand, plus le couple est grand.

Moment d'inertie (I)

Résistance d'un objet à la rotation : $I = \sum m_i r_i^2$. Dépend de la **répartition** de la masse par rapport à l'axe. Analogue de la masse en rotation.

Vitesse angulaire (ω)

Dérivée de l'orientation par rapport au temps (rad/s). Analogue de la vitesse linéaire.

Accélération angulaire (α)

Dérivée de la vitesse angulaire : $\vec{\alpha} = \frac{\vec{\tau}}{I}$. Analogue de $\vec{a} = \frac{\vec{F}}{m}$.

Moment angulaire (L)

$\vec{L} = I \cdot \vec{\omega}$. Se conserve en l'absence de couple externe — d'où le patineur qui accélère en rapprochant ses bras.

Théorème de Huygens-Steiner

$I = I_{\text{cm}} + md^2$. Permet de calculer I autour d'un axe parallèle à distance d à partir de I au centre de masse.

Roulement sans glissement

Condition $v_G = \omega R$: le point de contact est immobile par rapport au sol. La boule avance à la vitesse exacte de sa rotation.

Friction de glissement

Force cinétique au point de contact qui **ralentit** la translation **et accélère** la rotation, jusqu'à atteindre le roulement pur.

Résistance au roulement

Très faible force due à la déformation microscopique du sol. Décélère lentement une boule en roulement pur.

Gimbal lock

Perte d'un degré de liberté des angles d'Euler quand deux axes s'alignent. Évité par les **quaternions**.

Quaternion

Représentation d'une rotation 3D : $q = (w, x, y, z)$. Pas de gimbal lock, interpolation naturelle.
Intégré via $q_{\text{new}} = q_{\text{delta}} \cdot q_{\text{old}}$.

Orbites & Gravitation (Session 12)

Gravitation universelle

$\vec{F} = G \cdot \frac{m_1 m_2}{r^2} \cdot \hat{u}$. L'accélération subie ne dépend que de la masse attractive M et de la distance r .

Vitesse orbitale

$v = \sqrt{G \frac{M}{r}}$. Vitesse tangentielle pour une orbite **circulaire** parfaite à la distance r .

N-body

Gravité **mutuelle** entre tous les corps. Les orbites ne sont plus parfaitement circulaires — les planètes se perturbent.

Moteurs physiques commerciaux (Sessions 13–14)

Moteur physique commercial

Bibliothèque éprouvée (PhysX, Havok, Jolt, Box2D, Rapier) qui gère broad phase, solveur, sleeping, joints. Économise des années de R&D.

Rigid Body vs Soft Body

Rigid : indéformable (95 % des cas). **Soft** : déformable (tissu, gelée), simulé par masse-ressort ou FEM — beaucoup plus coûteux.

CCD (Continuous Collision Detection)

Teste la collision le long de la **trajectoire** entre deux frames, pas seulement aux positions discrètes. Évite le **tunneling** des objets rapides.

Tunneling

Un objet rapide traverse un mur mince entre deux frames : aucune position discrète n'est **dans** le mur. Le CCD le prévient.

Sleeping

Un objet immobile (vitesses sous un seuil) arrête d'être simulé. Économise du CPU. Se réveille s'il est touché ou reçoit une force.

Dynamic / Fixed / Kinematic

Dynamic : subit forces et collisions. **Fixed** : jamais bougé (murs, sol). **Kinematic** : contrôlé par le code, pousse les dynamic mais ignore les forces.

Raycast

Projette un rayon dans le monde et retourne le premier collider touché, avec distance et normale. Outil de **requête** — ne modifie pas la simulation.

Shapecast

“Si je déplace cette forme de A à B, que touche-t-elle en premier ?” Utilisé par les character controllers.

Overlap

“Quels corps chevauchent ce volume ?” Pour zones de dégâts, pickups, déclencheurs.

Déterminisme

Mêmes inputs → exactement mêmes outputs, même sur des machines différentes. Essentiel pour le multijoueur lockstep et les replays. Difficile (ordre des ops, flottants, threads).

ECS (Entity-Component-System)

Architecture des moteurs modernes : **Entity** (ID), **Component** (données pures), **System** (logique sur toutes les entités ayant certains composants). Composition, cache-friendly.

Rapier

Moteur physique écrit en Rust, compilé en WASM. Standard pour Three.js. API moderne, déterministe localement.

Jolt Physics

Moteur open-source C++ moderne, multithreadé. Moteur par défaut de Godot 4.4+. Rivalise avec PhysX/Havok.

WASM (WebAssembly)

Format binaire qui tourne à pleine vitesse dans le navigateur. Rapier est compilé en WASM ; chargement asynchrone via `RAPIER.init()`.

Joins, Moteurs & Raycasts (Session 15)

Joint (articulation)

Contrainte qui relie **deux** rigid bodies et limite leurs degrés de liberté relatifs. Résolue par le solveur au même titre que les contacts.

Fixed joint

Soudure : les deux corps sont rigidement liés (distance et orientation figées). Cassable à l'exécution — outil de la destruction.

Revolute joint

Charnière : les deux corps ne peuvent que **tourner** autour d'un axe. Porte, roue, coude.

Prismatic joint

Tiroir : les deux corps ne peuvent que **glisser** le long d'un axe. Vérin, piston, platine.

Spherical joint

Rotule : les ancres coïncident, rotations libres dans les 3 axes. Épaule, hanche, chaîne.

Motor (moteur d'articulation)

Composant optionnel d'un joint qui **pousse** vers une cible : **vitesse** (régulateur) ou **position** (servo). Paramètres stiffness et damping comme un ressort.

Limits (butées)

Bornes min/max sur l'angle (revolute) ou la translation (prismatic) d'un joint. Équivalent d'un butoir mécanique.

Contraintes & IK (Session 16)

PBD vs Impulse Based

PBD corrige les positions (simple, stable, vitesses incorrectes). **Impulse Based** calcule les impulsions (physiquement correct, conserve la quantité de mouvement).

PGS (Projected Gauss-Seidel)

Solveur itératif : pour chaque contact, calculer l'impulsion, l'appliquer, recommencer. Typiquement 4–8 itérations.

Warm Starting

Réutiliser les impulsions de la frame précédente pour converger plus vite.

Position Correction (Baumgarte)

Corriger légèrement la position pour réduire la pénétration : $\text{correction} = \text{penetration} \cdot \text{facteur} \cdot \vec{n}$.

Forward Kinematics (FK)

Des angles des articulations $\rightarrow$ position de l'extrémité. $\vec{p} = f(\theta_1, \theta_2, \dots)$.

Inverse Kinematics (IK)

De la position cible de l'extrémité $\rightarrow$ angles des articulations. $\theta = f^{-1}(\vec{p})$. Plusieurs solutions possibles, parfois aucune.

CCD (Cyclic Coordinate Descent)

IK itérative : ajuster les articulations une par une, de l'extrémité vers la base. Simple, converge vite, mouvements peu naturels.

FABRIK

IK par positions : aller (forward) de la main vers l'épaule, puis revenir (backward) en respectant les distances. Mouvements naturels, rapide.

Véhicules (Session 17)

Raycast Vehicle

Modèle arcade : pas de vraies roues, mais des **raycasts** depuis le châssis vers le sol. Plus stable, plus performant.

Suspension

Chaque roue simule un ressort : raycast $\rightarrow$ compression $c = 1 - \frac{d}{\text{maxLen}}$ $\rightarrow$ force $F = kc - \text{damping} \cdot v_{\text{susp}}$.

Grip / Slip angle

Grip : adhérence du pneu. **Slip angle** : angle entre la direction du pneu et la direction réelle du mouvement. Petit = tourne normalement, grand = dérape (drift).

Personnages (Session 18)

Dynamic Character Controller (DCC)

Personnage = Rigidbody standard. Interagit naturellement avec le monde mais difficile à contrôler (glisse, est poussé).

Kinematic Character Controller (KCC)

Personnage = Rigidbody kinematic. Contrôle total et game feel précis, mais il faut coder les collisions, la gravité, les pentes. Norme AAA.

Move and Slide

Algorithme : calculer le mouvement souhaité, tenter de déplacer, si on touche un mur **glisser** le long en annulant la composante normale : $\vec{v}' = \vec{v} - (\vec{v} \cdot \vec{n})\vec{n}$.

Corner Trap

Blocage quand on glisse le long de deux murs (un coin). Solution : traiter une collision à la fois, trier par profondeur, résoudre la plus profonde.

Coyote Time

Après avoir quitté une plateforme, le joueur peut encore sauter pendant ~ 0.1 s. Tolérance qui améliore le game feel.

Jump Buffering

Si le joueur appuie sur saut juste avant d'atterrir, le saut s'exécute dès l'atterrissage. Évite la frustration des sauts "ratés".

Physique, GPU & IA (Session 19)

Reinforcement Learning (RL)

Boucle : **Agent** (personnage) – **Environment** (monde physique) – **State** (capteurs) – **Action** (forces/torques) – **Reward** (note). Apprend par essai/erreur.

Neuroevolution

Algorithmes génétiques appliqués aux réseaux de neurones : population aléatoire → évaluation (fitness) → sélection → crossover/mutation → nouvelle génération.

Compute Shader

Programme qui s'exécute sur le GPU mais n'est **pas** un shader de rendu. Met à jour positions/vitesses/lifetime des particules stockées dans un SSBO.

SSBO (Shader Storage Buffer Object)

Buffer GPU accessible en lecture/écriture par un compute shader. Les données des particules y restent – pas de transfert CPU/GPU.